项目 7 避障数据获取

1、 获取激光雷达点云

Go2 机器狗开机会自动启动激光雷达的相关程序，并将激光雷达采集的点云和建立的环境高度地图发布到 DDS 话题。

1.1、 点云的消息话题

激光雷达默认发布两个点云话题：

激光雷达的原始点云：话题名称为 "rt/utlidar/cloud"，其所在坐标系为激光雷达坐标系 "utlidar_lidar"；

去除运动畸变后的点云：话题名称为 "rt/utlidar/cloud_deskewed"，其所在坐标系为与机身里程计一致的世界坐标系 "odom"。

注意：

在去除运动畸变时，我们使用机身里程计和 IMU 数据（由话题 "**rt/sportmodestate**"发布），将一帧点云中的所有点由其所在的不同时刻的激光雷达坐标系变换到了同一时刻的世界坐标系下。该时间戳默认与一帧点云的最后一个点的采集时间戳一致。

1.2、 点云的消息类型

点云的 IDL 消息格式为 PointCloud2_.idl，其与 ROS2 的点云

消息格式 PointCloud2.msg 定义一致。

PointCloud2_.idl 是一种用于表示三维点云数据的消息类型。PointCloud2 消息定义了一个包含点云数据的多维数组，其中每个元素都包含一个或多个字段。每个字段可以是一种预定义类型，如 **float32**、**int32** 等，也可以是自定义的类型。每个元素的大小和排列方式都是可以定义的。

PointCloud2_.idl 消息格式的定义及说明如下：

```
// generated from rosidl_generator_dds_idl/resource/idl.idl.em
// with input from sensor_msgs:msg/PointCloud2.idl
// generated code does not contain a copyright notice
#include "PointField_.idl"
#include "Header_.idl"
#ifndef __sensor_msgs__msg__point_cloud2__idl__
#define __sensor_msgs__msg__point_cloud2__idl__
module sensor_msgs {
module msg {
module dds_ {
struct PointCloud2_ {
```

```
        // Message header (including time of sensor data acquisition,
and the coordinate frame ID for 3d points).
        std_msgs::msg::dds_::Header_ header;
        // 2D structure of the point cloud.
        // If the cloud is unordered, height is 1 and width is the length of
the point cloud.
        unsigned long height;
        unsigned long width;
        // Describes the channels and their layout in the binary data blob.
        sequence<sensor_msgs::msg::dds_::PointField_> fields;
        // data
        boolean is_bigendian;          // Is this data bigendian?
        unsigned long point_step;     // Length of a point in bytes
        unsigned long row_step;        // Length of a row in bytes
        sequence<octet> data;              // Actual point data, size is
(row_step*height)
        boolean is_dense;              // True if there are no invalid points
    };
```

```
};  // module dds_

};  // module msg

};  // module sensor_msgs

#endif  // __sensor_msgs__msg__point_cloud2__idl__
```

header：一个标准的 ROS 消息头，包含了一些元数据，如时间戳、坐标系等。

height 和 **width**：分别表示点云数据在垂直和水平方向上的尺寸。如果点云是无序点云，则 height=1，width=点云的点数。

fields：PointField 类型的数组，表示每个元素包含的字段及其类型和偏移量。

is_bigendian：表示数据是否采用大端字节序，

point_step：表示每个元素的大小，

row_step：表示每一行数据的大小，

data：表示实际的点云数据。

1.3、 案例：使用 DDS 获取点云

在使用 DDS 订阅处理点云前，请先确保已安装 **unitree_sdk2**。如您还未安装，请先阅读快速开始文档。

在如下示例程序中，我们订阅了 DDS 点云话

题 "**rt/utlidar/cloud**"，并在回调函数中打印点云相关信息。

示例程序如下：

```
/**
 * @file subscribe_pointcloud.cpp
 * @brief Subscribe the pointcloud published from DDS topic
 * @date 2023-11-23
 */
#include <unitree/robot/channel/channel_subscriber.hpp>
#include <unitree/common/time/time_tool.hpp>
#include <unitree/idl/ros2/PointCloud2_.hpp>
#define TOPIC_CLOUD "rt/utlidar/cloud"
using namespace unitree::robot;
using namespace unitree::common;
void Handler(const void *message)
{
    const sensor_msgs::msg::dds_::PointCloud2_ *cloud_msg =
(const sensor_msgs::msg::dds_::PointCloud2_ *)message;
    std::cout << "Received a raw cloud here!"
```

```
                    <<          "\n\tstamp          =          "          <<
cloud_msg->header().stamp().sec()          <<          "."          <<
cloud_msg->header().stamp().nanosec()
                    << "\n\tframe = " << cloud_msg->header().frame_id()
                    << "\n\tpoints number = " << cloud_msg->width()
                    << std::endl
                    << std::endl;
    }
    int main(int argc, const char **argv)
    {
        if (argc < 2)
        {
            std::cout << "Usage: " << argv[0] << " networkInterface" <<
std::endl;
            exit(-1);
        }
        ChannelFactory::Instance()->Init(0, argv[1]);
        ChannelSubscriber<sensor_msgs::msg::dds_::PointCloud2_>
```

```
subscriber(TOPIC_CLOUD);
        subscriber.InitChannel(Handler);
        while (true)
        {
            sleep(10);
        }
        return 0;
    }
```

CMakeLists.txt 配置如下：

```
    project(helloworld)
    cmake_minimum_required(VERSION 3.5)
    SET(CMAKE_CXX_STANDARD 17)
    include_directories(/usr/local/include/ddscxx
/usr/local/include/iceoryx/v2.0.2)
    link_libraries(unitree_sdk2 ddsc ddscxx rt pthread)
    add_executable(subscribe_pointcloud
example/lidar/subscribe_pointcloud.cpp)
```

运行结果如下：

```
$ ./subscribe_pointcloud
Received a raw cloud here!
   stamp = 1683713030.572689533
   frame = utlidar_lidar
   points number = 1402
Received a raw cloud here!
   stamp = 1683713030.640552759
   frame = utlidar_lidar
   points number = 1423
Received a raw cloud here!
   stamp = 1683713030.705702066
   frame = utlidar_lidar
   points number = 1393
Received a raw cloud here!
   stamp = 1683713030.773043155
   frame = utlidar_lidar
   points number = 1413
...
```

1.4、 案例：使用 ROS2 可视化点云

在安装有 ROS2 的 Ubuntu 系统下，还可以运行 rviz2 工具来可视化当前点云。

步骤如下:

首先，运行如下命令 rviz2，打开 rviz2。

然后，在【Fixed Frame】处设置坐标系为 "odom"。

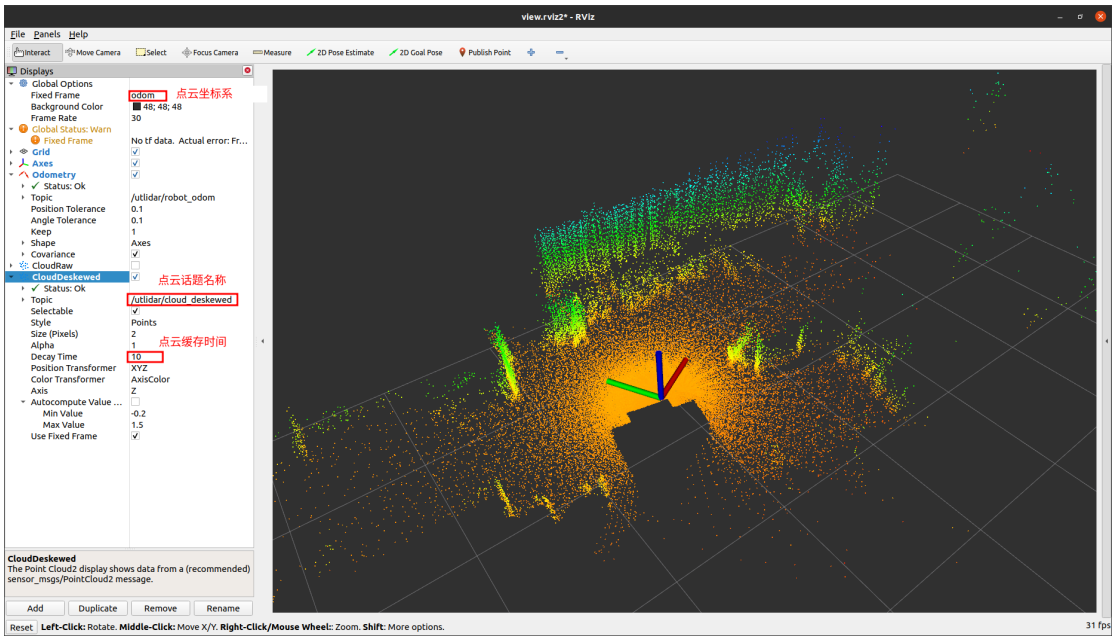
点击左下角【Add】，选择【By topic】，再选中对应的点云话题 "/utlidar/cloud_deskewed"。

可以设置【Decay Time】等于 1s 或更长时间，来缓存一定时间的点云数据。

注意:

DDS 下的话题名称前缀 rt 在 ROS2 下订阅时会自动消失，例如 DDS 下的话题"**rt/utlidar/cloud_deskewed**"，在 ROS2 下的话题名称会自动变为 "**/utlidar/cloud_deskewed**"。

rviz2 显示效果如下:



2、 获取高度地图

利用激光雷达去畸变后的点云和机器人里程计等数据，我们还建立了以机器人为中心的表示环境高度信息的高度地图，其消息话题为"**rt/utlidar/height_map_array**"，其所在坐标系为与机身里程计一致的世界坐标系 "**odom**"。该地图可以用于机器人避障、落足规划等功能。

2.1、 高度地图的消息格式

点云的 IDL 消息格式为 **HeightMap_.idl**，该消息格式的定义及说明如下：

```
// generated from rosidl_generator_dds_idl/resource/idl.idl.em

// with input from sensor_msgs:msg/PointCloud2.idl
```

```
// generated code does not contain a copyright notice
#ifndef __unitree_go_msgs__height_map__idl__
#define __unitree_go_msgs__height_map__idl__
module unitree_go {
module msg {
module dds_ {
struct HeightMap_ {
  // Header
  double stamp;              // timestamp
  string frame_id;           // world frame id
  // Map info
  float resolution;          // The map resolution [m/cell]
  unsigned long width;       // Map width along x-axis [cells]
  unsigned long height;      // Map height alonge y-axis [cells]
  float origin[2];           // Map frame origin xy-position [m]
                             //   the xyz-axis direction of map frame
is aligned with the world frame
  // Map data, in x-major order, starting with [0,0], ending with
```

```
[width-1, height-1]
        // For a cell whose 2d-array-index is [ix, iy],
        //     its position in world frame is: [origin[0] + ix * resolution,
origin[1] + iy * resolution]
        //     its cell value is: data[width * iy + ix]
        sequence<float> data;
    };
    };   // module dds_
    };   // module msg
    };   // module unitree_go
    #endif   // __unitree_go_msgs__height_map__idl__
```

stamp：消息时间戳

frame_id：世界坐标系名称

resolution：地图分辨率

height 和 width ： 地图在 x 和 y 方向上的栅格数目

origin[2]：地图坐标系原点在世界坐标系下的 xy 位置

data：地图高度数据

2.2、 案例：使用 DDS 订阅高度地图

在使用 DDS 订阅处理高度地图前，请先确保已安装 **unitree_sdk2**。如您还未安装，请先阅读快速开始文档。

在如下示例程序中，我们订阅了 DDS 高度地图话题 "**rt/utlidar/height_map_array**"，并在回调函数中打印了获取到的高度地图的相关信息。

示例程序如下：

```
/**
 * @file subscribe_height_map.cpp
 * @brief Subscribe the height map published from DDS topic
 * @date 2023-11-23
 */
#include <unitree/robot/channel/channel_subscriber.hpp>
#include <unitree/common/time/time_tool.hpp>
#include <unitree/idl/go2/HeightMap_.hpp>
#define TOPIC_HEIGHTMAP "rt/utlidar/height_map_array"
using namespace unitree::robot;
using namespace unitree::common;
```

```
    void Handler(const void *message)
    {
        // print msg info
        const unitree_go::msg::dds_::HeightMap_ *map_msg = (const
unitree_go::msg::dds_::HeightMap_ *)message;
        std::cout << "Received a height map here!"
                  << std::setprecision(14)
                  << "\n\tstamp = " << map_msg->stamp()
                  << "\n\tframe = " << map_msg->frame_id()
                  << "\n\twidth = " << map_msg->width()
                  << "\n\theight = " << map_msg->height()
                  << "\n\tresolution = " << map_msg->resolution()
                  << std::endl << std::endl;
        // Parse a height map to a pointcloud in world frame
        std::vector<std::array<float, 3>> cloud; // the element of
std::array is x, y and z respectively
        int width = map_msg->width();
        int height = map_msg->height();
```

```
        float resolution = map_msg->resolution();
        float originX = map_msg->origin()[0];
        float originY = map_msg->origin()[1];
        int index;
        std::array<float, 3> pt;
        for (int iy = 0; iy< height; iy++){
          for (int ix = 0; ix< width; ix++){
            index = ix + width * iy;
            pt[2] = map_msg->data()[index];
            if (pt[2] == 1.0e9) { // skip empty cell value which is set to
1.0e9
              continue;
            }
            pt[0] = ix * resolution + originX;
            pt[1] = iy * resolution + originY;
            cloud.push_back(pt);
          }
        }
```

```
    }
    int main(int argc, const char **argv)
    {
        if (argc < 2)
        {
            std::cout << "Usage: " << argv[0] << " networkInterface" <<
std::endl;
            exit(-1);
        }
        ChannelFactory::Instance()->Init(0, argv[1]);
        ChannelSubscriber<unitree_go::msg::dds_::HeightMap_>
subscriber(TOPIC_HEIGHTMAP);
        subscriber.InitChannel(Handler);
        while (true)
        {
            sleep(10);
        }
```

```
    return 0;
}
```

CMakeLists.txt 配置如下：

```
project(helloworld)
cmake_minimum_required(VERSION 3.5)
SET(CMAKE_CXX_STANDARD 17)
include_directories(/usr/local/include/ddscxx
/usr/local/include/iceoryx/v2.0.2)
link_libraries(unitree_sdk2 ddsc ddscxx rt pthread)
add_executable(subscribe_height_map
example/lidar/subscribe_height_map.cpp)
```

运行结果如下：

```
$ ./subscribe_height_map
Received a height map msg here!
  stamp = 1683713537.840576896
  frame = odom
  width = 128
  height = 128
```

```
    resolution = 0.06
Received a height map msg here!
    stamp = 1683713537.927782453
    frame = odom
    width = 128
    height = 128
    resolution = 0.06
...
```

2.3、 获取简单障碍物距离

通过对激光雷达点云进行处理，我们生成了机器人前方、左侧和右侧三个方向的实时的障碍物距离信息，这可以用于机器人的简单避障需求。用户可以通过订阅相应的话题获取该障碍物距离信息。

发布该简单障碍物距离信息的 DDS 话题为："rt/utlidar/range_info"

2.4、 障碍物距离的消息格式

考虑到该障碍物距离信息仅包括前左右三个距离信息，以及与现有 ROS2 消息格式的兼容，我们这里使用的是与 ROS2 中的 geometry_msgs::PointStamped_ 一致的 IDL 消息类型—

— geometry_msgs::msg::dds_::PointStamped_，具体定义如下所示：

```
#include "Point_.idl"
#include "Header_.idl"
#ifndef __geometry_msgs__msg__point_stamped__idl__
#define __geometry_msgs__msg__point_stamped__idl__
// Default Topic: "rt/utlidar/range_info"
module geometry_msgs {
module msg {
module dds_ {
struct PointStamped_ {
std_msgs::msg::dds_::Header_ header;
geometry_msgs::msg::dds_::Point_ point; // x, y and z of this point stands for front, left and right range distance respectively
};
};  // module dds_
};  // module msg
};  // module geometry_msgs
```

```
#endif  // __geometry_msgs__msg__point_stamped__idl__
```

header：消息帧头

point：其 xyz 三个分量分别表示前方、左侧和右侧三个方向的障碍物到机器人坐标系原点的距离

2.5、 案例：使用 DDS 障碍物距离

在使用 DDS 订阅处理障碍物距离前，请先确保已安装 **unitree_sdk2**。

在如下示例程序中，我们订阅了 DDS 障碍物距离话题 "rt/utlidar/range_info"，并在回调函数中打印了获取到的障碍物距离的相关信息。

示例程序如下：

```
/**
 * @file subscribe_range_info.cpp
 * @brief Subscribe the range info msg published from DDS topic
 * @date 2023-11-23
 */
#include <unitree/robot/channel/channel_subscriber.hpp>
#include <unitree/common/time/time_tool.hpp>
```

```
#include <unitree/idl/ros2/PointStamped_.hpp>

#define TOPIC_RANGE_INFO "rt/utlidar/range_info"

using namespace unitree::robot;

using namespace unitree::common;

void Handler(const void *message)

{

  // print msg info

  const  geometry_msgs::msg::dds_::PointStamped_ *range_msg =
(const geometry_msgs::msg::dds_::PointStamped_ *)message;

    std::cout << "Received a range info message here!"

              <<          "\n\tstamp          =          "          <<
range_msg->header().stamp().sec()          <<          "."          <<
range_msg->header().stamp().nanosec()

              << "\n\tframe = " << range_msg->header().frame_id()

              << "\n\trange front = " << range_msg->point().x()

              << "\n\trange left = " << range_msg->point().y()

              << "\n\trange right = " << range_msg->point().z()

              << std::endl << std::endl;
```

```
    }
    int main(int argc, const char **argv)
    {
        if (argc < 2)
        {
            std::cout << "Usage: " << argv[0] << " networkInterface" <<
std::endl;
            exit(-1);
        }
        ChannelFactory::Instance()->Init(0, argv[1]);
        ChannelSubscriber<geometry_msgs::msg::dds_::PointStamped_>
subscriber(TOPIC_RANGE_INFO);
        subscriber.InitChannel(Handler);
        while (true)
        {
            sleep(10);
        }
        return 0;
```

```
}
```

CMakeLists.txt 配置如下：

```
project(helloworld)
cmake_minimum_required(VERSION 3.5)
SET(CMAKE_CXX_STANDARD 17)
include_directories(/usr/local/include/ddscxx
/usr/local/include/iceoryx/v2.0.2)
link_libraries(unitree_sdk2 ddsc ddscxx rt pthread)
add_executable(subscribe_range_info
example/lidar/subscribe_range_info.cpp)
```

运行结果如下：

```
$ ./subscribe_range_info
Received a range info message here!
  stamp = 1683716737.840576896
  frame = odom
  range front = 1.5
  range left = 0.7
```

```
range right = 2.3
```

3、 项目总结

通过本项目，我们不仅掌握了点云数据的获取和可视化技术，还成功获取了当前的高度地图与障碍物信息。这些项目建立是实现路径规划和自主导航的重要步骤，为我们后续的项目奠定了坚实的数据基础。